

Relatório Técnico - Parte Teórica

Entrega contínua de uma plataforma de pedidos em microsserviços: do Docker Compose ao Kubernetes

Aluno: Francisco Guilherme do Nascimento Costa | RA: 128984 | Disciplina: Cloud DevOps - Orchestrating Containers and Micro Services | Tema: Plataforma de pedidos em microsserviços (Loja Veloz)

1. Microsserviços e o papel do DevOps em ambientes cloud-native

A **arquitetura de microsserviços** divide uma aplicação em serviços pequenos, independentes e orientados a domínio de negócio, cada um com seu ciclo de build, deploy e escala, comunicando-se por APIs ou eventos. Na Loja Veloz, os domínios naturais são pedidos, pagamentos e estoque, atrás de um API Gateway. Os ganhos são escala seletiva (o serviço de pedidos cresce na campanha sem replicar o resto), isolamento de falhas e entregas independentes. O custo é complexidade distribuída: latência de rede, consistência entre serviços, versionamento de contratos e, principalmente, dificuldade de diagnóstico.

Esse custo só é pagável com **DevOps**: cultura e práticas que integram desenvolvimento e operações por meio de automação, infraestrutura como código, entrega contínua e telemetria. Em ambientes *cloud-native* (contêineres, orquestração, APIs declarativas), o engenheiro DevOps projeta o caminho do código até a produção, tornando-o repetível, auditável e seguro. Os princípios do *12-Factor App* (configuração no ambiente, processos *stateless*, logs como fluxo, *disposability*) orientam como cada serviço deve ser escrito para ser operado assim.

2. Containerização: Docker, Kubernetes e quando usar cada um

Containerização empacota a aplicação e suas dependências em uma imagem imutável que executa de forma isolada (namespaces e cgroups do kernel), compartilhando o kernel do host. Diferente de máquinas virtuais, é leve e inicia em segundos, eliminando o problema do “na minha máquina funciona”. **Docker** é a ferramenta para construir imagens (Dockerfile, *multi-stage*), executá-las e, com **Docker Compose**, descrever um ambiente multi-serviço (redes, volumes, variáveis) em um arquivo, subindo tudo com um comando.

Kubernetes não substitui o Docker: atua em outro nível. É um orquestrador que, dado o estado desejado declarado em manifests, mantém-no em um *cluster* de várias máquinas: agenda Pods, reinicia os que falham (*self-healing*), faz descoberta de serviços e balanceamento (Services), atualiza sem indisponibilidade (Deployments), escala automaticamente (HPA) e gerencia configuração e segredos (ConfigMaps/Secrets).

Critério	Docker / Compose	Kubernetes
Escopo	Um host; desenvolvimento, testes, sistemas pequenos	Cluster multi-nó; produção com alta disponibilidade
Escala e auto-recuperação	Manual / limitada	Automática (HPA, reschedule, probes)
Deploy sem downtime	Não nativo	Rolling update, rollback, canary/blue-green
Complexidade operacional	Baixa	Alta (compensada por automação)
Uso no projeto	Ambiente local padronizado	Produção (pico da campanha)

Regra prática adotada: Compose quando o objetivo é reprodutibilidade local e simplicidade; Kubernetes quando há requisitos de disponibilidade, escala elástica e deploys frequentes e seguros, como na campanha promocional da Loja Veloz.

3. Fundamentação teórica

Orquestração de contêineres. O modelo é declarativo e baseado em *reconciliação*: controladores comparam continuamente o estado atual com o desejado e agem para convergir. Conceitos centrais: Pod (menor unidade de deploy), Deployment/ReplicaSet (réplicas e atualizações), Service (endereço estável), Ingress (entrada externa), *probes* (readiness decide se recebe tráfego; liveness decide se reinicia), requests/limits (agendamento e proteção de recursos), NetworkPolicy e Pod Security (defesa em profundidade). Para deploys, *rolling* troca réplicas gradualmente; *blue/green* mantém dois ambientes completos e troca o tráfego de uma vez; *canary* direciona uma pequena fração do tráfego à nova versão e a amplia conforme as métricas.

CI/CD em ambientes distribuídos. A *integração contínua* valida cada mudança (lint, testes, build, varredura de vulnerabilidades); a *entrega/implantação contínua* promove artefatos imutáveis e versionados (imagem com tag por commit) pelos ambientes, com *gates* (aprovação, testes) e rollback. Com muitos serviços, cada

um deve ter pipeline independente e compatível com seus vizinhos; constrói-se uma vez e promove-se o mesmo artefato (*build once, deploy many*); segredos ficam no cofre da plataforma de CI, nunca no repositório. *GitOps* e IaC (Terraform) estendem a mesma lógica à infraestrutura.

Observabilidade. É a capacidade de entender o estado interno do sistema a partir de suas saídas, apoiada em três pilares: **métricas** (séries numéricas agregadas, baratas, ótimas para alertas: taxa de requisições, erros, latência - os *golden signals*); **logs** (eventos detalhados, estruturados em JSON e enviados a stdout); e **traces** (o caminho de uma requisição através dos serviços, composto por *spans* ligados por um *trace id* propagado nos cabeçalhos). Em microsserviços o tracing distribuído é o que responde “em qual serviço a requisição ficou lenta ou falhou?”. *OpenTelemetry* padroniza a instrumentação (independente de fornecedor); um *service mesh* como Istio adiciona telemetria, mTLS e gestão de tráfego sem alterar o código.

4. Principais decisões arquiteturais e justificativas

Decisão	Alternativas	Justificativa
Compose para dev, Kubernetes em prod	Somente Compose; VMs	Reprodutibilidade local barata; produção precisa de escala, self-healing e rollout seguro.
Rolling update (maxUnavailable 0) + rollback automático	Blue/green; canary	Elimina downtime sem duplicar infra (custo) e sem a complexidade de malha; blue/green/canary ficam como evolução para pagamentos.
HPA por CPU (70%); VPA só como recomendação	VPA ativo; KEDA	Serviços stateless escalam horizontalmente; VPA ativo reinicia Pods e conflita com HPA em CPU.
Sem mensageria no MVP; evento registrado em log	RabbitMQ / Kafka	Fluxo é curto e precisa de resposta imediata; mensageria aumenta operação. Saga com compensação cobre falhas; fila entra quando houver processamento assíncrono.
OpenTelemetry + Prometheus; service mesh adiado	Istio desde o início	Instrumentação no código entrega traces e métricas com custo menor; mesh (mTLS, canary) é o próximo passo.
Secrets/ConfigMaps + PSA restricted + NetworkPolicy	Variáveis no código; sem políticas	Config por ambiente (12-Factor), menor superfície de ataque e segmentação zero-trust entre serviços.
Terraform (VPC + EKS)	Console manual; CloudFormation	Infra reproduzível e revisável; ecossistema de módulos oficiais e neutralidade de fornecedor.

Referências

- Kubernetes Documentation. *Concepts: Workloads, Services, Configuration, Security*. <https://kubernetes.io/docs/>
- Docker Documentation. *Dockerfile best practices; Docker Compose*. <https://docs.docker.com/>
- Wiggins, A. *The Twelve-Factor App*. <https://12factor.net/>
- HashiCorp. *Terraform Documentation*. <https://developer.hashicorp.com/terraform/docs>
- GitHub Docs. *GitHub Actions*. <https://docs.github.com/actions>
- OpenTelemetry Documentation. <https://opentelemetry.io/docs/> | Istio Documentation. <https://istio.io/latest/docs/>
- Google Cloud Platform. *Online Boutique (microservices-demo)*. <https://github.com/GoogleCloudPlatform/microservices-demo>